

Generative AI for the Quantified Self

Reliable Innovative Systems

Abdul Woahab Ali Qosai, Adam Galla, Aleksandar Nikolaev Stavrev, Joyesh Vinod Meshram, Mihail Grigorov Gerginov, Nadeem Abdulrahman Abdullahi

Computer Science, CS-IT8, 2025-05





Department of Computer Science
Aalborg University
http://www.aau.dk

AALBORG UNIVERSITY
STUDENT REPORT

Title:
Generative AI for Quantified Self

Theme:
Reliable Innovative Systems

Project Period:
Spring Semester 2025

Project Group:
cs-25-dat-8-12

Participant(s):
Abdul Woahab Ali Qosai
Adam Galla
Aleksandar Nikolaev Stavrev
Joyesh Vinod Meshram
Mihail Grigorov Gerginov
Nadeem Abdulrahman Abdullahi

Supervisor(s):
Sujay Basavaraj Shalawadi

Page Numbers: 38

Date of Completion:
May 28, 2025

Abstract:

This project explores how a personalized chatbot for quantified self can enhance engagement among recreational runners and cyclists. Guided by interviews with six Strava users, we implemented a chatbot prototype that combines six togglable user-requested features which allows customization and personalization of the chatbot. These features included a memory feature which recalls previous inputs by users. Moreover, context-aware summarizations of users' Strava data, goal tracking, chart visualizations, customizable chatbot personas, a weather widget and injury prevention monitoring feature. For storing user data we used SQLite and SurrealDB. Moreover we used Retrieval Augmented Generation, Gemini-2.5 Pro and its ability to use agent tools to implement the features. Post-study findings indicate that features which increased the chatbot's knowledge of personal details about users improved their experience and engagement, compared to generic features.

The content of this report is freely available, but publication (with reference) may only be pursued due to agreement with the author.

Contents

1 Introduction **1**

2 Related Works **3**
2.1 Personal Informatics and Reflective Feedback 3
2.2 LLM-Based Personalized Coaching in Digital Health 4

3 Pre Study **6**
3.1 Participants 6
3.2 Pre-Study Interviews 6

4 Implementation **9**
4.1 Chat Service 9
4.1.1 Prompts & Tooling 9
4.1.2 Choice of LLM 10
4.2 Memory & Embedding Service 11
4.3 Data Stores 12
4.4 Prompt Chains 12
4.5 Data Pipeline 13
4.5.1 Data Import 14
4.5.2 Data Retrieval 14
4.6 Front-end 15

5 Post-Study **17**
5.1 Post-Study Interview 17

6 Results **19**

7 Discussion **22**
7.1 Technical Limitations 22
7.1.1 Large data sets 22
7.1.2 Agent Tools Unpredictability 23
7.1.3 Strava Integration 24

7.1.4 Chart Generation Reliability 24
7.2 Short Evaluation Period 24
7.3 Implications and Risks of LLM-Based Health Coaching 24
7.3.1 Hallucination and Bias 25
7.3.2 Privacy 25

8 Future Work and Improvements **26**

9 Conclusion **27**

Bibliography **29**

A All Identified Features **34**

B Comparison of Large Language Models (LLMs) **35**

C Full Flow Examples **37**

Chapter 1

Introduction

Wearable activity trackers (WATs) such as Apple Watch, Fitbit, Garmin, and Oura are now ubiquitous and enable continuous collection of health metrics such as heart rate variability, sleep patterns, and detailed training data for hundreds of millions of users worldwide. Meta analyses of randomized controlled trials show that these devices increase daily step counts and time spent in moderate to vigorous physical activity across diverse age groups and clinical populations [1].

The Quantified Self [2] movement has inspired research into personal informatics systems that guide users through the stages of data collection, data integration, reflection, and action. In the field of human-computer interaction, designers have derived guidelines by analyzing qualitative feedback from interviews and diary studies. Recent work shows that combining rich interview insights with simple usage logs such as feature use counts, session duration statistics, and user retention data strengthens the relevance of these design guidelines [3, 4].

Regular physical activity offers profound health benefits, yet more than one quarter of people worldwide do not meet recommended activity levels[5]. One-on-one coaching with a trained health professional remains the gold standard for behavior change [6] because it provides highly tailored guidance, but it is costly, limited in reach, and difficult to scale. To broaden access, researchers have turned to mobile-health solutions such as smartphone apps and wearable trackers that use built-in sensors (e.g., accelerometers, GPS, and optical heart-rate monitors) to track steps, distance, movement intensity, sleep, and key physiological signals. These low-cost, on-demand tools democratize continuous health monitoring and support active lifestyles.

Conversational agents and AI chatbots have emerged as a promising way to provide personalized health advice [7, 8, 9]. Recent advances in large language models (LLMs) such as GPT-4o [10] bring new capabilities for fitness support. These models can generate nuanced, context-aware responses in natural language, synthesize diverse data sources including sensor streams, user journals, and past conversations, and retain conversational memory over multiple sessions. At the same time, they pose risks since they may hallu-

cinate unsupported recommendations [11], inadvertently expose sensitive data, or adopt a patronizing tone [12] if not carefully guided. Mitigating these risks requires careful prompt design, system level guardrails and ongoing human oversight.

The self-tracking community is large, and spans a wide range of activities and sports. To keep the project focused and manageable, we decided to concentrate on a few specific sports. The selection process for these specific sports included factors like the availability of online datasets, availability of participants practicing those sports, and the platforms they used. Although many activities and sports could have served as valid focuses, personal interest also influenced our decision. In the end, we decided to focus on running and cycling.

Building on these foundations and this specific focus, our exploratory study implements a prototype of an AI powered fitness chatbot for runners and cyclists. We collected user-driven feature requirements through pre-study interviews with six participants, a one-week trial run, and post-study interviews and analysis.

We define two categories of user-driven features, based on their function. The first category, *personalization*, refers to features that directly or indirectly increase the amount of user specific context available to the LLM. The second category, *customization*, refers to application specific features which are simply requested by participants and do not contribute to the amount of personal context available to the LLM.

Through an iterative development process, we identified and refined a core set of six candidate features, ranging from chat memory notes and selectable conversation personas to dynamic visualizations and goal setting. Rather than testing a full theoretical model, we focus on building a viable prototype, discovering which features users expect and want, how they perform in daily use, and what unmet needs emerge in a real world fitness context. Specifically, we address the following research questions.

RQ 1: Does personalization help with engagement and satisfaction with chatbots in quantified self applications?

RQ 2: What features do users want the chatbot to have in order to improve personalization?

RQ 3: How can large language models support the integration and delivery of user requested personalization features in fitness chatbots?

Chapter 2

Related Works

The chapter will introduce relevant literature surrounding the Quantified Self (QS) movement, personal informatics systems, with a specific focus on Wearable Activity Trackers (WATs) and the emerging role of AI-based health chatbots. The QS movement, rooted in "self-knowledge through numbers," spurred early adoption of self-tracking. Modern WATs such as Fitbit and Apple Watch have automated and popularized this, with studies such as Ferguson et al. [1] demonstrating their efficacy in increasing physical activity by an average of 1800 steps/day and aiding weight reduction of 1 kg, with sustained effects lasting for a minimum of six months. However, while WATs excel at collecting data, users often face challenges in interpreting these data meaningfully and translating them into sustained behavior change.

2.1 Personal Informatics and Reflective Feedback

Personal informatics systems are systems that offer assistance when individuals collect significant data for the purpose of self-reflection and self-knowledge [13]. Through the aforementioned self-reflection, these tools aim to provide a deeper understanding of one's habits, behaviors, and patterns, incentivize behavioral change [14, 15, 16, 17, 18]. While the concept of the *self-improvement hypothesis* has been around for some time [19], it can still be challenging to actually implement in practice. The *personalization paradox* (a term coined by Jarno Koponen in 2015 [20]) causes users to be "forced" into what they describe initially (for example - their end goal), without much leeway for further customization and a lack of variation. Ontanon et al. [21] and Zhu et al. [22] showcase an example with Netflix inaccurately predicting a user's preference based on their viewing history, thus exclusively recommending content from that temporary preference. Additionally, personal informatics often yields insights that cannot be acted upon retrospectively [19]. Our work aims to offer direct access to sufficient personalization tools that allow users to further contextualize their current needs, preferences, and goals, translating said context into additional information for the Large Language Model (LLM).

2.2 LLM-Based Personalized Coaching in Digital Health

Health coaching is a tried, tested, and proven technique to induce behavioral change [23, 24, 25]. Both in-person and automated health coaching rely on establishing a partnership with their client and motivates them to adapt healthier habits and lifestyles while also providing feedback [26]. With more means to quantify data than ever before (smartwatches, applications such as Strava, Garmin Connect, etc.), coaches can use said data to provide in-depth information to their clients, further supporting their adaptation via personalized feedback [1, 27, 28]. However, both forms of coaching also have their drawbacks - in-person coaching can be expensive and inaccessible [29]. With the rise of LLMs such as ChatGPT, Gemini, and Claude, the implementation of AI-based chatbots for health coaching is a heavily touched upon topic with applications such as WHOOP [30] and Strava's Athlete Intelligence [31], as well as papers like GPTCoach [8] and works by Yakura et al. [7] and Chiu et al. [32] Nevertheless, especially the papers, work with a set of system prompts that are difficult to mutate and adapt to a client's needs. With the rise of tooling such as PydanticAI (source), people can develop solutions that can easily adjust parameters.

Many self-tracking platforms have begun to integrate LLMs to help users make better sense of their data and enhance the overall experience. WHOOP, Garmin, and Strava each took different approaches with the same core goal of supporting novice users: WHOOP introduced 'WHOOP Coach', a GPT-4-based chatbot that allows free-form dialogue for personalized insights, while Garmin [33] and Strava [31] offer AI-generated activity summaries without back and forth interaction. Early feedback on these beta features has been mixed[34, 35, 36]. Some WHOOP users praise the chatbot for speeding up analysis, while others complain about its lack of context and overly generic replies[37]. Garmin [38] and Strava [39] users similarly criticize the summaries for limited value or contextual inaccuracies. These findings suggest an opportunity for innovation. By conducting qualitative research on user unmet needs and shortcomings in current implementations, we can identify ways to refine LLM integrations and unlock their full potential to support sustained change in health behavior.

Building on this opportunity, academic research is actively exploring how to refine LLM integrations for more effective personalized health coaching. Jörke et al. (2025) [8] introduced GPTCoach, an LLM-powered physical activity chatbot designed to replicate core components of evidence-based coaching programs such as Stanford's Active Choices. GPTCoach integrates motivational interviewing strategies, uses user-provided health data from wearables, and creates a customized coaching experience. Through formative interviews and a lab study with 16 participants, the authors found that users perceived the system as supportive and its advice as highly personalized and actionable. Importantly, GPTCoach enabled reflection on motivation and obstacles, suggesting that LLMs can go beyond surface-level analysis of tracking data to engage with qualitative aspects of behaviour change.

Similarly, other research investigates how LLMs can enhance user reflection in personal

informatics. Strömel et al. (2024) [12] explore how LLMs can improve reflection in personal informatics systems by translating fitness tracker data into personalized narratives. In a study with 273 participants, the authors compared traditional chart-based data visualization with GPT-4-generated textual summaries and a combination of both. The findings reveal that LLM-generated narratives encourage deeper reflection, greater focused attention, and a stronger sense of reward compared to charts alone. Participants appreciated the engaging and contextual nature of textual feedback, but also expressed a desire for greater specificity and a less patronizing tone. The study emphasizes the potential of generative AI to bridge the gap between quantitative fitness metrics and users' qualitative experiences, supporting richer, more reflective interactions with self-tracking data.

Chapter 3

Pre Study

In recent years, wearable activity trackers (WATs) and self-recorded health data have attracted considerable research attention to support health and well-being. Health-focused chatbots have been evaluated in contexts examining features such as personas [40] and goal setting [41], and users consistently report that interventions tailored to their individual circumstances are more relevant and effective. In this report, we explore how large language models (LLMs), combined with self-tracking data from WATs, can enable personalized assistance through a fitness chatbot.

We evaluated our AI fitness chatbot in a one-week trial run with six fitness hobbyists. First, we conducted exploratory interviews to gather feature expectations and preferences. The participants then provided feedback throughout the iterative prototype development. Then, each participated in a one week trial of the system. Finally, we conducted post-trial interviews to capture perceptions of the chatbot and identify opportunities for more customization and personalization.

3.1 Participants

We interviewed six volunteers, all of whom were personal acquaintances of the research team and were recruited through informal word-of-mouth outreach. Their ages ranged from 22 to 29 years, and all identified as male (see Table 3.1 for details). All were amateur fitness hobbyists and regular users of WATs. All participants had basic knowledge of AI. Four of the participants were runners, and two were cyclists. All of the participants tracked their activity using Strava.

3.2 Pre-Study Interviews

In advance of the one-week trial run, we conducted exploratory unstructured interviews with all six participants to identify what they hoped a personalized fitness chatbot would

Table 3.1: Demographic profile of the exploratory-interview cohort.

Age	Gender	Profession
29	Male	Analyst
24	Male	Receptionist
22	Male	Student
24	Male	Shop Assistant
24	Male	Student
29	Male	Sales Assistant

deliver. Each interview began with the question to set the theme for the conversation.

> *"What kind of personalization features would you like to see in LLMs, which would help with your engagement with tools for self-health and activity data tracking?"*

Participants described their fitness routines, current wearables, and gaps they experience when interpreting their data. Some wanted adaptive feedback based on their goals, some wanted feedback on data while contextualizing weather, sleep, and nutrition, and others desired intuitive summaries of past performance, map/photo-based representations of routes, and social media features to connect with other users.

We gained insights from the interviews which were synthesized into an initial feature set, which then guided an iterative development process. Through this process, we finalized a list of features. Due to time and complexity constraints, we picked out specific features from the initial feature list A.1.

By the end of this pre-usage phase, we had arrived at a set of six features for the working prototype.(See Table 3.2).

Selected Feature	Description
Persona	Users choose and change the chatbot's personality.
Visualizations	The chatbot creates line charts, area charts, and bar charts from user data.
Goal inputs	Users input goals as text, The chatbot provides comparative feedback on recent performance.
Memory	Users record thoughts and notes, The chatbot recalls and incorporates them in responses.
Injury prevention monitoring	the chatbot flags running distance increases >10%, or cycling heart rate >10 bpm above weekly average.
Weather widget	The chatbot displays a weather forecast widget.

Table 3.2: Finalized features and their short descriptions.

To ensure clarity on what each feature is included in the prototype, we briefly describe their purpose and expected user benefits.

Persona Letting users create their own custom personas that allows them to tailor the tone and style of feedback to their motivational preferences. Some participants told us that they wanted a persona which was stern when they want to be pushed or encouraging when they need support, which would help sustain long-term engagement.

Visualizations On demand chart generation provides simple visualizations such as bar charts or line plots that show trends at a glance. Whether it is pace over time or a weekly mileage histogram, participants felt that visual context would help them spot patterns and anomalies more effectively than raw numbers.

Goal inputs By entering free text goals, such as "I want to run two kilometers in under 25 minutes" or "walk five thousand steps each day", users receive immediate comparative feedback on recent workouts versus their stated aspirations. This feature closes the feedback loop between planning and performance reinforcing progress or highlighting areas in need of extra focus.

Memory The AI can recall user provided notes such as how a particular route felt or a minor injury that flared up and weave these recollections into future advice.

Injury prevention monitoring Users can test when running mileage increases by more than ten percent from one week to the next or when cycling heart rate is more than ten beats per minute above their weekly average. The chatbot will then interpret and present the analysis and address concerns about overtraining.

Weather widget A built-in weather widget shows the upcoming forecast for the user's locale, enabling better planning. Some participants noted that weather can make or break a run, so having forecast information front and center felt useful.

Together these six features form our prototype that balances selected features with our implementation timeline, ensuring a focused yet personalized and customized fitness tracking experience during the trial run.

Chapter 4

Implementation

Guided by our pre-study interviews, we developed our fitness chatbot prototype. In this section, we will discuss the system architecture. Our code is available on Github [1].

4.1 Chat Service

The main service of the backend is the chat service, which communicates with the Large Language Models (LLMs) provider (Google Gemini [42] in this case) and specifies the prompts, system prompts or instructions, and tools used with the LLM. To facilitate communication with the Gemini API, we used PydanticAI [43], which simplified the definition of prompts, system prompts or instructions, and tools while providing the additional benefit of supporting multiple LLM vendors without requiring codebase rewrites. Within this service, we define instructions that inform the LLM about key properties, including the prompts that set basic rules defining custom behavior and information such as the current year, and how custom formats are structured, such as the data formats required for visualizations that the front-end uses to construct the final charts users see. Additionally, the defined tools allow the LLM to access relevant information through tool calls, such as retrieving user activity data when needed. In addition to communicating with the LLM provider, the chat service also communicates with the memory service, which allows it to access relevant memories that the user has uploaded. This enables the usage of RAG (Retrieval-Augmented Generation), which helps provide a more personal experience for the user as the LLM can recall previously uploaded information.

4.1.1 Prompts & Tooling

A system prompt is a prompt that is programmatically injected into an agent to steer the behavior of the LLM during response generation. It can be considered as a meta-instruction that defines properties such as role, tone, or capabilities of the assistant. Our

[1] `https://github.com/Quan-self`

approach utilizes PydanticAI tools that are dynamically adapting based on the needs of the agent.

Formally, a tool $\tau : X \to Y$ maps a structured input $x \in X \subseteq \mathbb{R}^n$ to a structured output $y \in Y \subseteq \mathbb{R}^m$, where X and Y are defined by the tool's input/output Pydantic models. The invocation of tools is mediated by function-calling protocols, and their results can be grounded in the subsequent response of the LLM.

In the presented architecture on Figure 4.1, tools act as bridges between natural language understanding and backend microservices, enforcing a contact-driven approach to augmenting LLM capabilities with deterministic logic and external data.

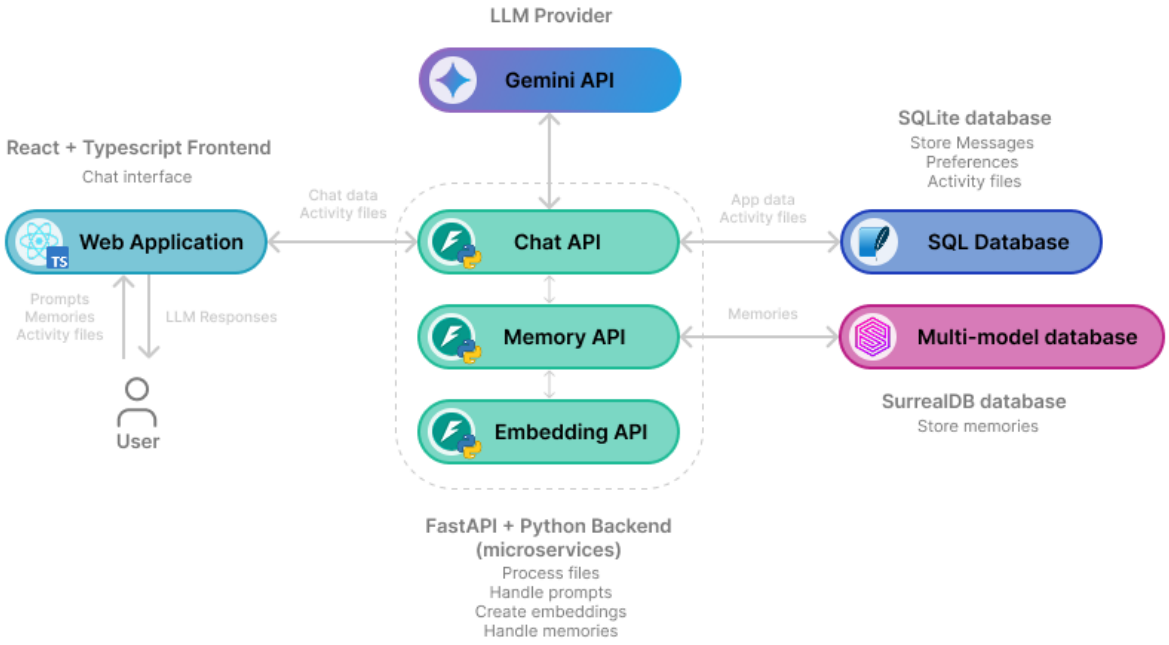


Figure 4.1: General Architecture of the Application

4.1.2 Choice of LLM

When choosing an LLM, we need to take into consideration the performance of the model together with its context length. Tables B.1 and B.2 showcase a comparison of different LLM models that were considered, alongside their features.

When choosing the LLM model, the most important consideration for us was a sufficiently long context length due to the breadth of the data that needed to be analyzed. For this reason, we decided to experiment with several models, including GPT-4.1 and Gemini 2.5 Pro, as they best fit this condition. To choose the model, we evaluated a range of features, such as data visualization generation and tool call usage, including activity context retrieval.

One of the main reasons we preferred Gemini 2.5 Pro over GPT-4.1 is because Gemini 2.5 Pro proved to be more well-rounded and generally performs better in tasks such as data visualization.

4.2 Memory & Embedding Service

The memory and embedding services were specifically created to facilitate semantic memory operations. The embedding service generates vector representations of text, enabling similarity-based searches between stored memories and user queries. This enables the system to retrieve relevant memories based on semantic meaning rather than exact keyword matching. These two services were separated out from the chat service due to the embedding model requiring large Python libraries like PyTorch [44] and the loading of embedding models that causes the application startup to slow down greatly.

Definition 1 (Embedding)
An **embedding** is a dense, real-valued vector representation of $\mathbf{v} \in \mathbb{R}^n$ of the user's and the model's textual input. This transformation encodes a context of text into a fixed-dimensional space such that semantically similar texts correspond to geometrically proximate vectors under similarity measures, such as cosine similarity. For a given text input t, the embedding function $f : T \to \mathbb{R}^n$ maps t to a vector $\mathbf{v} = f(t)$, preserving relationships.

Definition 2 (Memory)
A **memory** $m \in M$ is an indexed persistent representation of contextual information, which can afterward be incorporated into subsequent prompts to enhance relevance and continuity.

It is defined as a tuple $m = (t, \mathbf{v}, \sigma)$, where t is a textual unit (such as a message), $\mathbf{v} \in \mathbb{R}^n$ is its embedding vector generated by the embedding service, and σ is a set of metadata attributes (e.g. relevance score). By utilizing the above-mentioned similarity measures, the system can semantically align a user's current query q with the most suitable past content:

$$\text{Retrieve}(q) = \{m_i \in M : sim(f(q)), v_i) \geq \theta\}$$

where $f(q)$ is the embedding of the query and θ is a configured similarity threshold.

Definition 3 (Retrieval-Augmented Generation)
Retrieval-Augmented Generation (RAG) is a hybrid architecture that integrates neural language generation with information retrieval, allowing a language model to ground its outputs in external knowledge sources [45], thus enhancing the factual accuracy, specificity, and context-awareness of generated responses. Formally, a RAG pipeline consists of the following components:

- *Query Formation*: Given a user input q, a query vector $\mathbf{v}_q = f(q) \in \mathbb{R}^n$ is generated via an embedding model (in our case - provided by the embedding service (see definition 1)).

- *Retrieval Module*: A set of k relevant documents $\{d_1, \ldots, d_k\} \subseteq D$ is selected from a corpus D using similarity search (see Definition 2).
- *Augmented Prompt Construction*: The retrieved documents are concatenated and incorporated into the prompt p to form an augmented prompt $p' = [d_1, \ldots, d_k, q]$, which is then passed to the language model.
- *Response Generation*: A language model $\mathcal{L}$ generates a response r conditioned on p': $r = \mathcal{L}(p')$

4.3 Data Stores

To store user data such as messages, preferences, and activity data, SQLite was chosen due to its simplicity. SQLite does not require setting up a separate database server, making it easier to get up and running with local application development and requiring less setup. This enabled the creation of the foundation for the application's data store for the chat service, in combination with the SQLAlchemy object-relational mapping (ORM) package, which was used in the chat service to communicate with the SQLite file.

For the memory service, SQLite could not be used as it does not have the ability to store vectors without additional extensions. Therefore, a database with native support for vectors was needed. SurrealDB [46] was chosen for this purpose, as it has this capability and many more features as a multi-model database. This choice had the additional advantage that it could theoretically be updated later - if time had allowed - to explore other RAG solutions such as GraphRAG [47].

In theory, SurrealDB could have been used for all storage use cases, as it also supports relational data representation. However, this approach was not adopted due to the team's limited experience with SurrealDB compared to their existing familiarity with SQLite from previous projects.

4.4 Prompt Chains

In Figure 4.2, when a user opens the application and types a prompt, the application backend processes the user's input and checks which chat features, such as memory, charts, or goal, they have enabled. These features enrich the prompt and enable the LLM to produce a more fitting response that aligns with the user's preferences. This is achieved by appending guiding text to the prompt request.

After this, the message is forwarded to the response generation part and sent to the LLM where additional aspects, such as the user's customized persona and preferences, are added as instructions to the request. These instructions are included alongside the general base prompt, which contains information applicable to all requests (for example, guidance on choosing the appropriate tool).

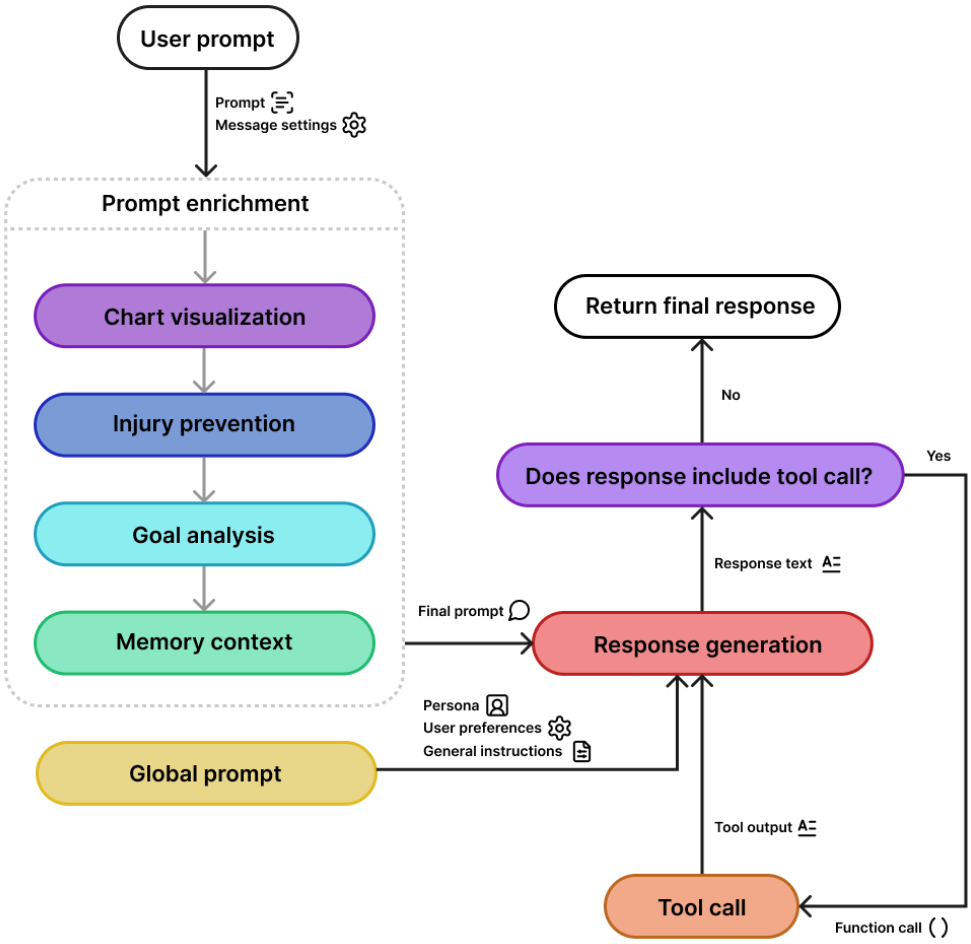


Figure 4.2: An example of a prompt chain

When the message reaches the LLM, it can either generate the final response directly or invoke a tool if it deems this necessary to fulfill the user's request. If no tool call is required, the final response is created by the LLM and streamed to the user. If a tool call is required, the LLM returns a specially crafted message to the chat service to run a predefined tool. For instance, if the user requests the chatbot to comment on their activity on the 15th March, the LLM must first retrieve that data in order to provide an answer grounded in reality. This process repeats until the LLM does not need to call any extra tools and generates the final response text.

4.5 Data Pipeline

We built our data pipeline with two design principles in mind. We aimed for an approach that would allow convenient import of data by participants without much manual work,

while also keeping the technical complexity of such a solution low. Due to certain limitations from the Strava API, a direct data integration was not possible and therefore, we decided to implement the second most convenient approach. This would require users to export their data from Strava in the form of `.csv` and `.fit` files, which would subsequently be uploaded to our backend through the web interface.

4.5.1 Data Import

Strava offers two scopes of exporting self-tracked data–profile and activity. Both scopes contain relevant information; therefore, our data pipeline had to support both types of exports. The profile export contains a csv file, where all user activities are listed with high-level information (e.g. total distance, max speed). A specific activity export doesn't support `.csv` files and has to be exported as a fit file containing detailed timestamped information (e.g. elevation, pace).

The pipeline starts by a user exporting all relevant data from Strava. First, the general information about all activities is uploaded in the form of a `.csv` file. The contents of the file are then mapped to a model, which is then parsed and stored to a SQLite database. After the general information is stored, the user uploads specific activity fit files. Each file is converted to a `.csv` format, which is subsequently cleaned to prevent missing or partial rows by merging rows with the same timestamp. If the file contains more than 500 rows, it passes through additional condensing logic, which thins the file further to match the limit of 500 rows. After the file is cleaned and ready to be stored, the general activities which were stored in the previous step are retrieved and filtered to find the general activity corresponding to the activity details file. The activity details file is then stored in a separate table and linked to the general activity through a foreign key.

4.5.2 Data Retrieval

When users start a new chat with the LLM, they are required to specify a timeframe they are interested in analyzing. Users can provide time in absolute or relative formats (e.g. *"last week"* or *"1st of May, 2025"*). This time reference is then parsed by the LLM using the `parse_time` tool call, which returns a precise time interval in datetime format. This datetime interval is then used by the LLM to retrieve activities which occurred during that time interval using the `get_activities_data` tool call. This tool call currently returns all general and specific data matching the time frame, which could get problematic, especially in cases where too much data is returned due to large timeframes or very active users. The problem lies in the risk of overflowing the LLM context window, and in a production-ready solution, certain checks would need to be implemented to prevent such cases from occurring.

4.6 Front-end

To allow easy distribution of the prototype to participants, we built the frontend as a web application using TypeScript with the React framework. The interface was built with Tailwind CSS in combination with the `shadcn/ui` component library, which allowed us to keep the interface components consistent across the application while giving us a high degree of customization and providing efficient component implementation.

The user interface was built with principles of simplicity, clarity, reliability, and customization in mind. We focused on maximizing the probability of a good user experience so that the users can simply focus on experiencing the features for self-tracking, without confusion or frustration due to an unclear, cluttered, or faulty user interface.

The interface features several segments, each concerned with a specific task. Figure 4.3 shows the default landing page, which invites the user to chat with the LLM. On the left side, there is a sidebar containing utilities, such as creating a new chat, history of conversations, and buttons for uploading data, entering the customization dialog, and logging out.

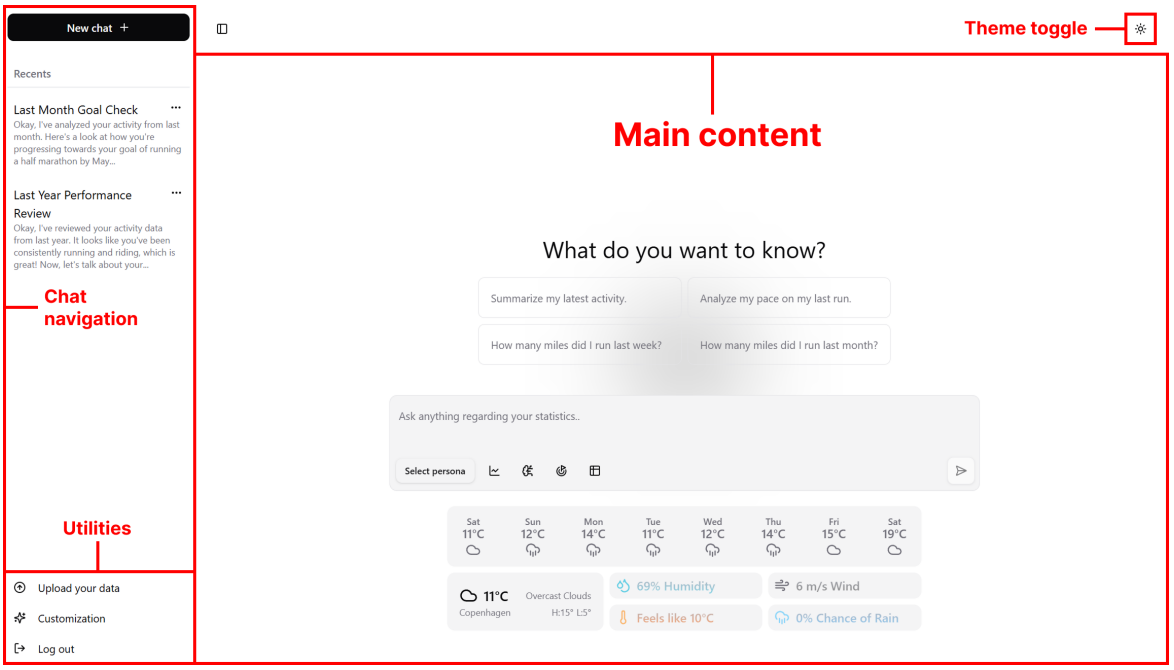


Figure 4.3: Main interface of the application

Figure 4.4 shows the customization dialog, which is the second most important component of our application after the chat itself. It gives users the ability to toggle features that either they requested, or try features which were requested by other participants. We decided to implement this customization logic to prevent initial overwhelming and confusion, and instead provide a clear way of slowly becoming familiar with features, one

by one. After a certain feature is enabled, another tab corresponding to that feature is displayed, where users can fine-tune or interact with the feature.

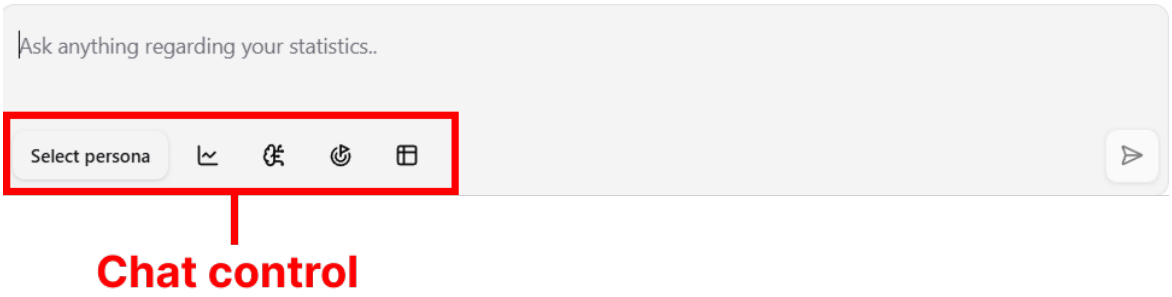


Figure 4.4: Feature toggles

Examples of the full flows can be found in Appendix C.

Chapter 5

Post-Study

5.1 Post-Study Interview

After the participants used the chatbot for a week, new meetings were arranged to question them about their experience with the chatbot. These meetings contained a semi-structured interview, containing the following questions (see Table 5.1):

#	Interview Questions
1	Looking back on your time with the chatbot, what worked well for you and what didn't?
2	Which parts of the chatbot experience did you find most helpful or engaging? Why?
3	Did you feel your chatbot experience was personalized? What stood out to you?
4	What aspects of the chatbot met or exceeded your expectations?
5	Which parts were frustrating or could be improved?
6	How did the features influence your motivation to continue using the chatbot?

Table 5.1: Post-interaction interview questions for chatbot users

The purpose of these questions was to identify where the chatbot solution fell short, and which part worked well. Moreover, the questions are designed to figure out which parts of the chatbot affected the users' experience positively, and if the ability to personalize the chatbot was noticeable, and how it affected the experience and engagement. The questions were also used to try to identify features which played the most important role for the user experience and engagement.

Furthermore, the use of these questions was also to figure out if we implemented the features to a satisfactory degree. This would be used for internal use, to figure out which features of the chatbot were lagging and would need to be improved for future work.

The implementations of the chatbot also included the recording of logs of usage. During the general questions listed in table 5.1. We used the recorded logs to figure out how

participants used the chatbot, and why they did certain actions during the usage period. The purpose of this was to get a better understanding of how users engaged with the chatbot. This information was to guide parts of the interview, as we asked users questions such as (see Table 5.2):

#	**Usage Questions**
1	Why did the user toggle/untoggle certain features?
2	Did they not find these features helpful?
3	Did the feature not work properly?
4	Why did the user only engage with a few messages on some days compared to others?
5	Were those few messages enough to get satisfactory responses from the chatbot?
6	Why did the user choose not to engage with the chatbot on some days? Was the experience with the chatbot bad, or were there other reasons?

Table 5.2: Questions for diagnosing user engagement patterns

These are some examples of how we used the logs to get a better understanding of the usage period. However, the different participants had different usage patterns, which made the questions regarding usage vary a lot. Ultimately, these logs gave us good insight regarding whether the participant found the chatbot satisfactory, which features were highly used compared to others, and how much the participants chose to personalize and customize the chatbot. Moreover, we gained information on whether users continued coming back to use the chatbot, and how much they engaged with it.

Chapter 6

Results

From reviewing the logs and interviewing the participants regarding the overall experience with the chatbot, the responses indicate that personalization increased the user experience and engagement. Participants commented that the ability to edit the persona of the chatbot increased their overall engagement. They also shared comments such as *"I liked that I could change the personality of the chatbot, and figure out what suited me best"*. All interviewees shared similar responses regarding the persona feature. Some of them, however, commented that sometimes the chatbot responses could become too generic, and the persona would not become as present as they hoped. This resulted in some participants trying to write long prompts into the "Persona" feature, which would make it better fit their needs in the short term. However, after engaging with the chatbot with multiple messages, the AI agent would sometimes dismiss the selected persona when responding and default to its original behavior. A participant commented: *"When I made a new persona, the chatbot would not use it very well in the responses I got, and it would forget the persona I had selected"*. After further testing this problem, we could identify the same issue the user had pointed out, and it appeared to be a consequence of the randomized nature of Large Language Models (LLMs). Nevertheless, the participants unanimously agreed that the persona feature contributed to an increased engagement of the chatbot.

Participants commented that the memory feature of the chatbot was particularly useful. And this feature seemed to really increase the personal experience of the chatbot. One of the participants from the interview was asked how he used the memory feature, and he responded that at one time he had inputted that he liked running in the area of the Aalborg tower in the evening as the area has a good view. When later asking the chatbot for suggestions regarding when and where to have his next exercise, the chatbot would recall that the user likes to run in that area in the evening. When asked how this feature affected his experience, the participant commented, *"I was actually surprised that it worked, and it further added to the experience"*. In addition, all participants seemed to respond positively to the memory feature, as they felt the chatbot knew them better and had a more personal understanding of them when engaging with it.

Some participants liked that the chatbot could generate visualizations based on their data and that they could customize different kinds of charts, such as area chart, line chart, bar chart. Some participants reported that this feature worked alright, but would sometimes require multiple prompts to generate the exact chart that they wished to display. When asked about whether this feature had special importance regarding their continued engagement, the responses seemed to indicate that this feature did not play a high importance. Moreover, a participant commented *"I didn't really need the feature, I only tried to generate a chart out of curiosity to see how it looked compared to Strava"*. Overall, from the logs and interviews, this feature seemed more as a "nice to have". Participants appreciated the option to generate a customized chart on demand, but from reviewing the logs, it was clear that it did not get as much user engagement as initially thought. The participants seemed to have alternatives when wanting to view their data as charts such as the Strava app, which did not require prompting and long wait times to generate. The friction and wait times regarding the generation of these charts seem to have contributed to the lack of usage.

From reviewing the logs, all users had turned on the external weather integration widget. All participants seem to appreciate the weather widget, but it did not play an importance regarding their engagement with the chatbot. However, it did give them a better user experience when using the chatbot, as they could see an easy forecast for the week when thinking about their next exercise. However, the idea and ability to customize different widgets to display on the chatbot seemed to resonate with the participants. One participant mentioned, *"It was nice to have a weather forecast in the chatbot, but I think it could be better if I had the option to choose between more widgets to display"*. When asking the participant what other widgets could be useful, the participant mentioned integration with his Google Calendar, and the ability for the AI agent to use the calendar to help schedule workouts in the future. When discussing this widget with the participant, he seemed very confident that a calendar widget and the ability of the AI agent to use his calendar to schedule future workouts would definitely have increased his engagement with the chatbot. Overall, participants seemed positive on the ability to customize the chatbot with widgets, and participants seemed to wish for a greater selection of widgets to choose from.

The participants also enjoyed the Goal input feature. They would use the Goal feature to input some desired outcome from their workouts. When asking one of the participants how he used this feature, he mentioned, *"I tried to input that I would like to run 10km in less than 1 hour and 30 minutes. After one of my workouts I would try to ask the chatbot how close I am to achieving my goal"*. The participant mentioned that this feature worked well, and the chatbot used his latest run to compare with his goal. The chatbot summarized the latest run, and gave him his avg. pace, and the pace he should aim for. The chatbot could also distinguish between his moving pace and the avg. pace including stops and breaks. The chatbot told him he was very close to achieving his goal, if he kept his moving pace and avoided breaks. Overall, the ability for the participants to include specific goals and have the chatbot respond based on their workout seemed to have contributed positively to their

overall engagement and experience.

Participants also enjoyed that the chatbot was good at retrieving the data that they wanted summarized, and was essential for their engagement with the chatbot. However, frustration was expressed regarding users needing to re-upload data after each workout, as the chatbot was not directly connected with their Strava accounts.

The injury prevention monitoring feature had been used by some of the participants. One of the participants mentioned that it activated at some point when he used it, which was nice. However, he would have liked the ability to customize the behavior of the feature. Moreover, some participants seemed to have liked the idea of defining triggers themselves, and specifying when they should activate, such as when a participant accomplishes an all-time high with regard to some metric. And such a feature would have contributed to higher engagement.

From the results of the interviews, we noticed that features that help the chatbot understand the participants better. The ability for the participants to customize the persona of the chatbot to fit their needs, seemed to have contributed the most to their engagement with the chatbot. These are also the features that seemed to affect the users most positively. Moreover, our qualitative analysis indicates that personalization of chatbots for quantified self does increase engagement and user experience, especially features that give the chatbot the ability to know personal details about the users. However, the perfect implementation of these features seems difficult, as a consequence of the unpredictable behavior of LLMs.

Chapter 7

Discussion

The initial goal of this study, as reflected through the research questions, is related to identifying and evaluating personalization features. However, during the pre-study, participants suggested a number of features which fell outside of this scope, due to the fact that they didn't directly or indirectly contribute to the amount of user-specific context provided to the Large Language Model (LLM). These are categorized as customization, because they are requested by participants, and are therefore helping to customize their user experience with the chatbot. Due to such features arising in the interviews, we decided to expand the scope of the project to include an analysis of application-specific features in regards to the effect on user engagement and satisfaction. We believe that this allowed us to capture a more complete picture of possible user expectations in the real-world fitness chatbot interactions.

7.1 Technical Limitations

It is important to consider the limitations when agreeing on any given technology. Everything comes with advantages and disadvantages. For this project, the benefits of our chosen technologies have largely benefited us and allowed us to pursue and develop a solution. However, it is equally important to examine the constraints these technologies come with. We looked into some key challenges such as managing data too large to fit within the AI's context window, addressing unpredictable behavior from the AI agent, and navigating difficulties in external integrations.

7.1.1 Large data sets

LLMs have improved drastically in a very short amount of time. Despite their rapid progression, a persistent issue remains in terms of their finite context window. This means the amount of tokens, the text, a model can process and retain in its memory at any given time. No matter how we approach this, it is a scaling issue and will always remain a bot-

tleneck when handling large amounts of text. In prolonged conversations with an LLM, it becomes increasingly harder for the AI to take into account certain information or properly prioritize what to mention. For example, in extended conversations, it can become harder for the LLM to recall details or accurately prioritize which pieces of information are most relevant to the ongoing conversation.

A known problem is working with large documents or datasets, which is exactly the case here. If we attempt to feed all the available data for all activities of the user, it would simply be denied by the LLM service due to the exceeded token limit. Further concerns arise when working with paid services, as the cost of using so many more tokens can make the product impractical.

These types of problems can be anticipated and considered beforehand, whether the benefit is worth it or not. This was taken into account when designing the architecture and is reflected in the backend services and the database (see Figure 4.1). Using a database allows us to store a lot of user information and implement various solutions to help the AI query that data. It can be a challenge to know exactly which data to retrieve. Conveniently, in this case, it was possible to relate the users to their activities and create a relational database schema to allow for custom queries. Since it would be inconsiderate to require the user to provide a very specific activity ID, we decided to perform the search based on dates. Here it was important to provide the AI with a context of time so expressions relative to the current moment such as *"yesterday"* or *"last Monday"* can be translated into actual dates. The limited context continues to be an issue, especially with multiple activities per day. Another optimization is to compress the provided data. Out of the box, the devices used to capture these metrics are quite precise and can measure every second. Cleaning up this data and averaging it over a larger time span such as a minute can greatly reduce the amount of tokens.

7.1.2 Agent Tools Unpredictability

As mentioned earlier, we used agent tools to allow the LLM to better understand the context and act appropriately when specific situations arise. These tools are predefined methods or functions that the LLM is instructed to use. The prompt given to the LLM outlines when and how to call these tools, along with the expected parameters it should provide. It is a very fluid system with no strict rules, as you cannot exactly predict if the AI is going to use the agent tool or not. This turned out to be a problem, as there were instances where the LLM would not call the tool even if the situation required it. This would result in the AI not being able to understand the current date, failing to find an activity even when it is in the database, or getting only an overview of the activity instead of detailed data. Although it is possible to iteratively modify and refine the LLM prompt to minimize these errors and guide it on how to use the tools more effectively, it remains impossible to guarantee perfectly consistent and correct responses every time.

7.1.3 Strava Integration

Integrating with an active platform the users use, such as Strava, was one of the original goals. However, it turned out to be less likely due to the scope of the project and the strict requirements needed for that to happen. While Strava provides an extensive API that is capable of providing the needed functionality, it also requires a strict approval process. Developing an integration with the Strava API was a straightforward task and was implemented early on. This would allow users to more easily work with the system by directly authenticating with Strava instead of manually downloading and uploading data files. However, upon further research, this integration was available only to a single user in total. It would have to undergo an approval process in order to get published, which would take time, require a public website, and other regulations. Strava explicitly disallows the usage of the API for AI [48], and this project might fall in that category. Furthermore, the platform forbids leveraging its data for analytics and customer-insight generation, which our project aims to provide. Overall, such integration would benefit the users, but it was not feasible for this project.

7.1.4 Chart Generation Reliability

The current implementation of the chart generation relies on the LLM to correctly create the structure of the chart with the correct data. This design allows for more flexibility as the LLM can handle more specific edge cases. However, this is also prone to errors due to the probabilistic nature of LLMs. As previously mentioned, hallucinations can lead to wrong data being used in the visualizations. It would be more reliable if we instead fetch the data and use a standard chart visualization service as we already predefine the graph types we want.

7.2 Short Evaluation Period

From the interviews, it seemed like if the users had more time to test the chatbot, it might have changed their perception. Participants seem to agree that they initially conducted a lot of experiments when granted access to the chatbot, in order to test the different features and modify them to figure out how the application worked, and how to make it suit them the best.

7.3 Implications and Risks of LLM-Based Health Coaching

LLMs do offer great potential benefits. However, as with all disruptive technologies, it is important to consider the risks associated with implementation and regular use. In this section, we discuss some of the potential risk factors related to LLMs, especially in regard to their integration with self-tracked and health-related data.

7.3.1 Hallucination and Bias

LLM hallucinations may cause a number of issues like poor user experience and misinformation. While safeguards such as prompt engineering, output filtering and retrieval-augmented generation (RAG) [45] can reduce the frequency of hallucinations, they cannot fully eliminate them due to the inherent probabilistic and generative nature of LLMs. Hallucinations may cause frustration when the user is knowledgeable in the discussed topic. However, more importantly, they pose the threat of misinforming the user. This can be especially harmful in the fitness and health setting, as incorrect information presented as an objective fact could lead the user into self-harming actions. One way to mitigate hallucinations is through RAG. RAG works by introducing an explicit information-retrieval step prior to text synthesis. Grounding the generation process in verifiable external evidence constrains the model's latent associations, thereby reducing the likelihood of fabricating unsupported claims while simultaneously enabling transparent source attribution and easier knowledge updating. Aside from implementing prevention techniques such as RAG, it is important to inform the users about the potential incorrect information that can be provided to them by an LLM.

LLMs sometimes return biased responses which can either discriminate against certain groups or misinform in the same way as hallucinations due to the nature of the data they are trained on. Minimizing bias can be achieved during training and finetuning, or with techniques such as output filtering or prompt engineering.

7.3.2 Privacy

Health related data is a sensitive form of information which must be treated carefully due to the high potential for misuse. We stored all fitness data on a secure virtual private server (VPS) which is hosted in the EU, with all participant consent in order to comply with the General Data Protection Regulation (GDPR). Furthermore, due to the fact that our system used the Gemini LLM provider, users were informed and agreed to share their data with this provider. Gemini does collect, and may use the data for training only when using the free trial account. Since our application leveraged a Gemini model not available to the free tier Gemini accounts, users were required to set up billing with free credits and therefore, as per Gemini's privacy policy, the data was not used for training. Google does retain the prompts, outputs and contextual information for fifty-five days for abuse monitoring[49].

Chapter 8

Future Work and Improvements

From the interviews, we also collected suggestions for improvement and further development. These do not include the features that were originally identified by the users but weren't selected to be implemented in the prototype A.1. However, these features were identified in the post-study.

- **Summarize workout based on locations:** The ability of users to ask the chatbot to summarize a workout based on a location, instead of date.
- **Add loading indicator:** Provide an indicator to let users know the chatbot is thinking and not yet finished responding.
- **Integration with existing health platforms:** Enable automatic access to users' latest activity data from health platforms without manual uploads.
- **Calendar integration:** Allow users to connect their calendar so the chatbot can schedule and plan future workouts.
- **Better persona implementation:** Ensure that the chatbot understands and consistently maintains user-defined personas throughout the conversation.
- **Integration with traffic:** Use external traffic data to suggest optimal workout routes.
- **Mobile support:** Improve accessibility by providing a user-friendly mobile interface for the chatbot.
- **Past conversations as context:** Leverage previous conversations as context when engaging with users.

Chapter 9

Conclusion

This qualitative study investigated how personalized Large Language Models (LLM)-based fitness chatbots impact user experience and engagement for quantified self. This research employed pre-study interviews, building a prototype with six personalization and customization features, a one-week trial run, and post-study interviews to answer questions regarding personalization and desired features. The findings give great insight into how personalized AI can enhance the user experience and engagement.

RQ 1: Does personalization help with engagement and satisfaction with chatbots in quantified self applications?

RQ 2: What features do users want the chatbot to have in order to improve personalization?

RQ 3: How can large language models support the integration and delivery of user requested personalization features in fitness chatbots?

The results of our research clearly indicate that personalization enhances user interaction and satisfaction with fitness chatbots. Users reported that features allowing the chatbot to be personalized according to their individual interactions and preferences make the experience engaging. The qualitative analysis highlighted that personalization made the chatbot more personal. Specifically, persona customization of the chatbot and the addition of the memory feature were most frequently reported as contributors to increased engagement. Our evaluation confirms that when users perceived the chatbot as being more mindful of their own data and preferences, their experience and engagement improved.

From the pre and post study, we concluded that features that increase the chatbot's knowledge of personal details relating to a participant were very positively received, and the participants agreed that these features increased their engagement. Moreover, participants suggested improvements such as customizing triggers that would activate based on certain metrics. Such a feature is supported by our conclusion, as it modifies the chatbot to behave in a personal way. Furthermore, a suggestion for calendar integration, which

gives the chatbot abilities to schedule future workouts based on the participant's schedules, supports the claim that features that make the chatbot more knowledgeable of the person using it are favored.

The support of integrating user-requested personalization features with chatbots can be done through, providing context to the chatbot in the form of prompts. Moreover, the implementation of agent tools which can upload user activity data to a database and vice versa, retrieve it. This grounds the LLM response based on the user's personal information. Moreover, the implementation of retrieval argument generation (RAG) and embedding models allows applications to do semantic search and retrieve past input from participants and use these inputs as context when responding to the users. Furthermore, including user inputs and prompt enrichment can make chatbots behave in user specified ways, and therefore increase personalization. However, the perfect implementation of these features can be hard to achieve, as a consequence of the unpredictable nature of LLMs.

In summary, our findings demonstrate that incorporating personalized LLM-driven features, especially those that let the chatbot understand personal details and adopt a custom persona, can meaningfully boost engagement and satisfaction with quantified self chatbots.

Bibliography

[1] T. Ferguson et al. "Effectiveness of wearable activity trackers to increase physical activity and improve health: a systematic review of systematic reviews and meta-analyses". In: *The Lancet Digital Health* 4.8 (Aug. 2022). PMID: 35868813, e615–e626. DOI: 10.1016/S2589-7500(22)00111-X.

[2] Melanie Swan. "The quantified self: Fundamental disruption in big data science and biological discovery". In: *Big data* 1.2 (2013), pp. 85–99.

[3] Frank Bentley et al. "Three hours a day: Understanding current teen practices of smartphone application use". In: *arXiv preprint arXiv:1510.05192* (2015).

[4] Junze Li et al. "DiaryHelper: Exploring the Use of an Automatic Contextual Information Recording Agent for Elicitation Diary Study". In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 2024, pp. 1–16.

[5] The World Health Organization (WHO). *Nearly 1.8 billion adults at risk of disease from not doing enough physical activity*. https://www.who.int/news/item/26-06-2024-nearly-1.8-billion-adults-at-risk-of-disease-from-not-doing-enough-physical-activity.

[6] Timothy W Bickmore, Daniel Schulman, and Candace L Sidner. "A reusable framework for health counseling dialogue systems based on a behavioral medicine ontology". In: *Journal of biomedical informatics* 44.2 (2011), pp. 183–197.

[7] Riku Arakawa and Hiromu Yakura. "Coaching Copilot: Blended Form of an LLM-Powered Chatbot and a Human Coach to Effectively Support Self-Reflection for Leadership Growth". In: *Proceedings of the 6th ACM Conference on Conversational User Interfaces*. New York, NY, USA: Association for Computing Machinery, 2024. URL: https://doi.org/10.1145/3640794.3665549.

[8] Matthew Jörke et al. "GPTCoach: Towards LLM-Based Physical Activity Coaching". In: *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. CHI '25. ACM, Apr. 2025, pp. 1–46. DOI: 10.1145/3706598.3713819. URL: http://dx.doi.org/10.1145/3706598.3713819.

[9] Mahyar Abbasian et al. "Conversational health agents: A personalized llm-powered agent framework". In: *arXiv preprint arXiv:2310.02374* (2023).

[10] OpenAI. *GPT 4o*. 2024. URL: https://openai.com/index/hello-gpt-4o/.

[11] Muhammad Aurangzeb Ahmad, Ilker Yaramis, and Taposh Dutta Roy. "Creating trustworthy llms: Dealing with hallucinations in healthcare ai". In: *arXiv preprint arXiv:2311.01463* (2023).

[12] Konstantin R. Strömel et al. "Narrating Fitness: Leveraging Large Language Models for Reflective Fitness Tracker Data Interpretation". In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. CHI '24. New York, NY, USA: Association for Computing Machinery, 2024. DOI: 10.1145/3613904.3642032. URL: https://doi.org/10.1145/3613904.3642032.

[13] Ian Li, Anind Dey, and Jodi Forlizzi. "A stage-based model of personal informatics systems". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 557–566. ISBN: 9781605589299. DOI: 10.1145/1753326.1753409. URL: https://doi.org/10.1145/1753326.1753409.

[14] Marit Bentvelzen et al. "Revisiting Reflection in HCI: Four Design Resources for Technologies that Support Reflection". In: *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 6.1 (Mar. 2022). DOI: 10.1145/3517233. URL: https://doi.org/10.1145/3517233.

[15] Elizabeth Victoria Eikey et al. "Beyond self-reflection: introducing the concept of rumination in personal informatics". In: *Personal and Ubiquitous Computing* 25.3 (2021), p. 601. DOI: 10.1007/s00779-021-01573-w.

[16] Chris Elsden, David Kirk, and Abigail Durrant. "A Quantified Past: Toward Design for Remembering With Personal Informatics". In: *Human-Computer Interaction* 31 (Sept. 2015), p. 150923205236008. DOI: 10.1080/07370024.2015.1093422.

[17] Izabelle F Janzen and Joanna McGrenere. "Reflective Spring Cleaning: Using Personal Informatics to Support Infrequent Notification Personalization". In: *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. CHI '22. New York, NY, USA: Association for Computing Machinery, 2022. ISBN: 9781450391573. DOI: 10.1145/3491102.3517493. URL: https://doi.org/10.1145/3491102.3517493.

[18] Amid Ayobi et al. "Flexible and Mindful Self-Tracking: Design Implications from Paper Bullet Journals". In: *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. CHI '18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 1–14. ISBN: 9781450356206. DOI: 10.1145/3173574.3173602. URL: https://doi.org/10.1145/3173574.3173602.

[19] Elisabeth Kersten - van Dijk et al. "Personal Informatics, Self-Insight, and Behavior Change: A Critical Review of Current Literature". In: *Human-Computer Interaction* 32 (Jan. 2016). DOI: 10.1080/07370024.2016.1276456.

[20] Jarno M Koponen. *The Future Of Algorithmic Personalization | TechCrunch*. 2015. URL: https://techcrunch.com/2015/06/25/the-future-of-algorithmic-personalization/.

[21] Santiago Ontanon and Jichen Zhu. "The Personalization Paradox: the Conflict between Accurate User Models and Personalized Adaptive Systems". In: *Companion Proceedings of the 26th International Conference on Intelligent User Interfaces*. IUI '21 Companion. New York, NY, USA: Association for Computing Machinery, 2021, pp. 64–66. ISBN: 9781450380188. DOI: 10.1145/3397482.3450734. URL: https://doi.org/10.1145/3397482.3450734.

[22] Jichen Zhu et al. "Personalization Paradox in Behavior Change Apps: Lessons from a Social Comparison-Based Personalized App for Physical Activity". In: *Proc. ACM Hum.-Comput. Interact.* 5.CSCW1 (Apr. 2021). DOI: 10.1145/3449190. URL: https://doi.org/10.1145/3449190.

[23] Seonuk An and Rhayun Song. "Effects of health coaching on behavioral modification among adults with cardiovascular risk factors: Systematic review and meta-analysis". In: *Patient Education and Counseling* 103.10 (2020), pp. 2029–2038. ISSN: 0738-3991. DOI: https://doi.org/10.1016/j.pec.2020.04.029. URL: https://www.sciencedirect.com/science/article/pii/S0738399120302378.

[24] Leigh Ann Simmons and Ruth Q. Wolever. "Integrative Health Coaching and Motivational Interviewing: Synergistic Approaches to Behavior Change in Healthcare". In: *Global Advances in Health and Medicine* 2.4 (2013). PMID: 24416683, pp. 28–35. DOI: 10.7453/gahmj.2013.037. eprint: https://doi.org/10.7453/gahmj.2013.037. URL: https://doi.org/10.7453/gahmj.2013.037.

[25] David H. Thom et al. "A Qualitative Study of How Health Coaches Support Patients in Making Health-Related Decisions and Behavioral Changes". In: *The Annals of Family Medicine* 14.6 (2016), pp. 509–516. ISSN: 1544-1709. DOI: 10.1370/afm.1988. eprint: https://www.annfammed.org/content/14/6/509.full.pdf. URL: https://www.annfammed.org/content/14/6/509.

[26] Jeanette Olsen. "Health Coaching: A Concept Analysis". In: *Nursing forum* 49 (Mar. 2014), pp. 18–29. DOI: 10.1111/nuf.12042.

[27] C. F. Chung et al. "More Than Telemonitoring: Health Provider Use and Nonuse of Life-Log Data in Irritable Bowel Syndrome and Weight Management". In: *Journal of Medical Internet Research* 17.8 (2015), e203. DOI: 10.2196/jmir.4364. URL: https://doi.org/10.2196/jmir.4364.

[28] H. Rutjes et al. "The Influence of Personal Health Data on the Health Coaching Process". In: *Frontiers in Big Data* 5 (2022), p. 678061. DOI: 10.3389/fdata.2022.678061. URL: https://doi.org/10.3389/fdata.2022.678061.

[29] Elliot G. Mitchell et al. "Automated vs. Human Health Coaching: Exploring Participant and Practitioner Experiences". In: *Proc. ACM Hum.-Comput. Interact.* 5.CSCW1 (Apr. 2021). DOI: 10.1145/3449173. URL: https://doi.org/10.1145/3449173.

[30] WHOOP. *Introducing WHOOP Coach, Powered By OpenAI*. Accessed: 2025-05-23. Sept. 2023. URL: https://www.whoop.com/us/en/thelocker/introducing-whoop-coach-powered-by-openai/.

[31] Strava. *Strava's Athlete Intelligence Translates Workout Data into Simple and Personalized Insights*. Accessed: 2025-05-23. Oct. 2024. URL: https://press.strava.com/articles/stravas-athlete-intelligence-translates-workout-data-into-simple-and.

[32] Yu Ying Chiu et al. *A Computational Framework for Behavioral Assessment of LLM Therapists*. 2024. arXiv: 2401.00820 [cs.CL]. URL: https://arxiv.org/abs/2401.00820.

[33] Garmin. *Elevate your health and fitness goals with Garmin Connect+ | Garmin Newsroom*. Garmin Newsroom, Mar. 2025. URL: https://www.garmin.com/en-US/newsroom/press-release/wearables-health/elevate-your-health-and-fitness-goals-with-garmin-connect/ (visited on 04/29/2025).

[34] u/tigerdata. *Has anyone found a valuable use case for Whoop coach?* https://www.reddit.com/r/whoop/comments/17rqged/has_anyone_found_a_valuable_use_case_for_whoop/. Reddit post. (Visited on 04/29/2025).

[35] u/potatololx. *Whoop Coach is a game changer.* https://www.reddit.com/r/whoop/comments/173k1of/whoop_coach_is_a_game_changer/. Reddit post. (Visited on 04/29/2025).

[36] u/salihveseli. *Totally worth the subscription. Active Intelligence is truly next level!* https://www.reddit.com/r/Garmin/comments/1jt7cmg/totally_worth_the_subscription_active/. Reddit post. (Visited on 04/29/2025).

[37] u/pyepush. *Whoop claims to be a "coach" but it really doesn't do any coaching.* https://www.reddit.com/r/whoop/comments/14dg7tj/whoop_claims_to_be_a_coach_but_it_really_doesnt/. Reddit post. (Visited on 04/29/2025).

[38] u/rigit84. *Garmin's Active Intelligence feels useless.* https://www.reddit.com/r/Garmin/comments/1jsdn0m/garmins_active_intelligence_feels_useless/. Reddit post. (Visited on 04/29/2025).

[39] Leo Schwartz. *Fitness app Strava has a new AI coach—and its wacky comments are going viral*. Yahoo Finance, Oct. 2024. URL: https://finance.yahoo.com/news/fitness-app-strava-ai-coach-110000542.html (visited on 04/27/2025).

[40] Marcia Nißen et al. "The effects of health care chatbot personas with different social roles on the client-chatbot bond and usage intentions: development of a design codebook and web-based study". In: *Journal of medical Internet research* 24.4 (2022), e32630.

[41] Abhishek Aggarwal et al. "Artificial intelligence–based chatbots for promoting health behavioral changes: systematic review". In: *Journal of medical Internet research* 25 (2023), e40789.

[42] Google. *Gemini 2.5 Pro: Best for coding and complex prompts*. 2025. URL: https://deepmind.google/models/gemini/pro/.

[43] The Pydantic Team. *PydanticAI: Agent Framework / shim to use Pydantic with LLMs*. 2024. URL: https://ai.pydantic.dev/.

[44] A Paszke. "Pytorch: An imperative style, high-performance deep learning library". In: *arXiv preprint arXiv:1912.01703* (2019).

[45] Yunfan Gao et al. "Retrieval-augmented generation for large language models: A survey". In: *arXiv preprint arXiv:2312.10997* 2.1 (2023).

[46] SurrealDB. *SQLAlchemy*. URL: https://surrealdb.com/.

[47] Darren Edge et al. "From local to global: A graph rag approach to query-focused summarization". In: *arXiv preprint arXiv:2404.16130* (2024).

[48] Strava. *Strava API Agreement*. 2024. URL: https://www.strava.com/legal/api.

[49] Google. *Gemini API Usage Policies*. 2025. URL: https://ai.google.dev/gemini-api/docs/usage-policies.

Appendix A

All Identified Features

All Identified Features	Description
Persona	Users choose and change the chatbot's personality.
Correlation between sleep and performance	The ability to upload sleep data, and have the AI agent analyze users' sleep data and how it affects their performance.
Visualizations	The chatbot creates line charts, area charts, and bar charts from user data.
Goal inputs	Users input goals as text, The chatbot provides comparative feedback on recent performance.
Memory	Users record thoughts and notes, The chatbot recalls and incorporates them in responses.
Injury prevention monitoring	The chatbot flags running distance increases >10%, or cycling heart rate >10 bpm above weekly average.
Personalized route discovery	Enable the AI agent to suggest new routes based on the user's vicinity.
User-defined route preferences	The AI agent provides route suggestions based on user-defined preferences such as distance, elevation, and scenery.
Map/Photo-based representation of routes	The AI agent presents suggested routes on a map with real photos of the route.
Connect with friends	The platform can access other users' data to showcase their activities, completed workouts, and routes, and create challenges between friends.
Weather widget	The chatbot displays a weather forecast widget.

Table A.1: Identified Features and Their Descriptions

Appendix B

Comparison of Large Language Models (LLMs)

Table B.1: Comparison of different LLMs (part 1)

Model	Key Strengths	Context Window	Notable Features
Claude 4 Opus	Advanced reasoning, coding, tool use	200K tokens	"Extended thinking" mode; excels in long-form tasks; superior coding capabilities.
GPT-4o	Multimodal generation, conversational AI	128K tokens	Enhanced image generation; improved emotional resonance in outputs.
GPT-4.1	Superior coding, instruction following, long context	1M tokens	Excels in coding tasks; improved instruction adherence; supports extensive context windows.
Gemini 2.5 Pro	Deep reasoning, coding, multimodal processing	1M tokens	"Deep Think" mode; excels in complex tasks; integrated into various Google services.
Mistral Medium 3	Cost-effective, enterprise adaptability	128K tokens	High performance at lower cost; suitable for self-hosted environments.
DeepSeek V3	Open-source, efficient reasoning	128K tokens	Competitive performance with lower computational requirements.

Table B.2: Comparison of different LLMs (part 2)

Model	Cost Efficiency	Input Price (/ 1M tokens)	Output Price (/1M tokens)
Claude 4 Opus	Moderate	$15.00	$75
GPT-4o	Moderate	$2.50	$10.00
GPT-4.1	Moderate	$2.00	$8.00
Gemini 2.5 Pro	Moderate	$1.25 ($\leq$ 200k tokens), $2.50 ($>$ 200k tokens)	$10.00 ($\leq$ 200k tokens), $15.00 ($>$ 200k tokens)
Mistral Medium 3	High	$0.40	$2.00
DeepSeek V3	High	$0.27 (cache miss), $0.07 (cache hit)	$1.10 (cache miss), $0.55 (cache hit)

Appendix C

Full Flow Examples

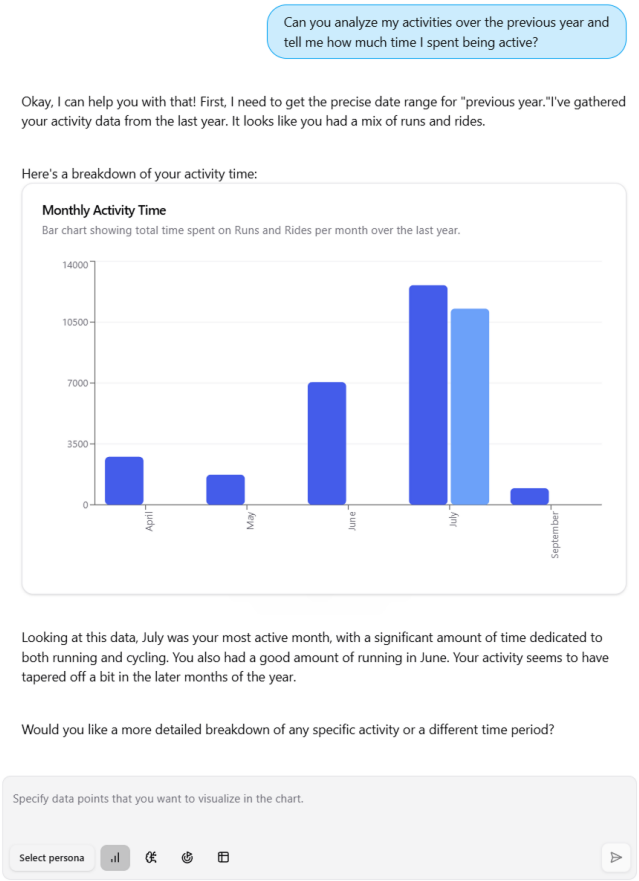


Figure C.1: Full chat example #1

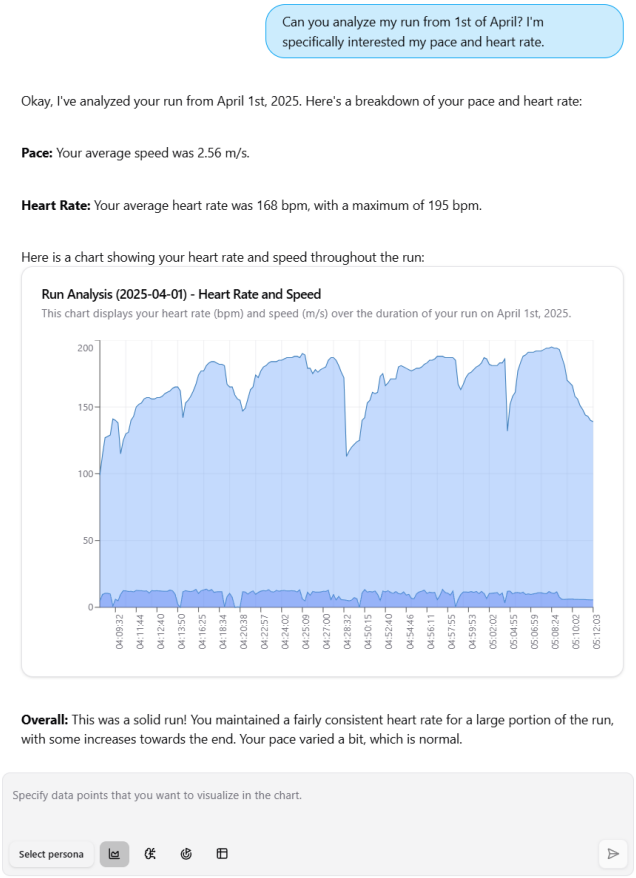


Figure C.2: Full chat example #2